

Automatisierter Sicherheitsbericht

Erstellt am: 17.05.2026 14:53:50

1. Management-Zusammenfassung

Im vorliegenden automatisierten Scan der Anwendung `http://juice-shop:3000` wurden mehrere Seiten einer Single-Page-Application untersucht. Dabei wurden drei sicherheitsrelevante Schwachstellen identifiziert, von denen zwei als hochkritisch bis kritisch einzustufen sind:

1. SQL-Injection mit Login-Bypass auf der Login-Funktion
2. Cross-Site-Scripting (XSS) über die Suchfunktion
3. Fehlender Schutz gegen Brute-Force-Angriffe auf der Login-Seite

Die gravierendste Feststellung ist die SQL-Injection in der Authentifizierung, da sie laut Rohdaten bereits zu einem erfolgreichen Login als Admin-Benutzer geführt hat. Dies bedeutet ein unmittelbares Risiko für die vollständige Kompromittierung der Anwendung und aller darin verarbeiteten Daten.

Die XSS-Schwachstelle in der Suche erlaubt die Ausführung von eingebrachtem JavaScript im Browser. Je nach Kontext kann dies zum Diebstahl von Sitzungen, zur Manipulation von Inhalten oder zur Ausführung von Aktionen im Namen eines Benutzers führen.

Zusätzlich wurde festgestellt, dass nach 40 fehlgeschlagenen Login-Versuchen keine erkennbare Schutzmaßnahme greift. Das erhöht das Risiko für Passwort-Angriffe erheblich.

Mehrere weitere Seiten und Formulare wurden erkannt, aber in den Rohdaten wurden dort keine bestätigten Schwachstellen dokumentiert. Die Formulardaten liefern nur Kontext und sind kein Beleg für eine tatsächliche Schwachstelle.

2. Technischer Überblick über die analysierten Seiten

Der Scanlauf war zum Zeitpunkt der gelieferten Metadaten noch als `running` markiert. Die vorliegenden Daten enthalten jedoch bereits verwertbare Ergebnisse.

Analysierte Ziele / Seiten

Untersucht wurden unter anderem:

- `/#/`
- `/#/about`
- `/#/login`
- `/#/contact`

- `/#/photo-wall`
- `/#/score-board`
- `/#/forgot-password`
- `/#/register`
- `/#/complain`
- `/#/chatbot`
- `/#/deluxe-membership`
- `/#/search?...`
- `/ftp/legal.md`

Erkannte Formulare / Eingabebereiche

Es wurden mehrere Formulare bzw. SPA-Form-Kandidaten erkannt, darunter:

- Login-Formular mit E-Mail, Passwort, "Remember me"
- Passwort-zurücksetzen-Formular
- Registrierungsformular
- Kontaktformular mit CAPTCHA
- Beschwerdeformular inkl. Dateiuupload
- Photo-Wall-Upload inkl. Dateiuupload und Bildbeschreibung
- Chatbot-Eingabefeld
- Suchfelder auf mehreren Seiten

Allgemeine Header-Beobachtungen

Wiederkehrende HTTP-Header:

- `access-control-allow-origin: \*`
- `x-content-type-options: nosniff`
- `x-frame-options: SAMEORIGIN`

Diese Header sind teilweise positiv, insbesondere `nosniff` und `SAMEORIGIN`.

Der Header `Access-Control-Allow-Origin: \*` fällt auf, ist aber allein anhand der Rohdaten nicht als eigenständige Schwachstelle belegbar, da wesentliche Informationen fehlen, etwa:

- welche Ressourcen damit freigegeben werden,
- ob Authentifizierungsdaten übertragen werden,
- ob browserseitig sicherheitsrelevante Kombinationen vorliegen.

Daher wird dies hier nur als technische Beobachtung, nicht als bestätigte Schwachstelle gewertet.

3. Priorisierte Liste der gefundenen Schwachstellen

1. SQL-Injection mit erfolgreichem Login-Bypass

- Betroffene URL: `http://juice-shop:3000/#/login`
- Risiko: Sehr hoch / kritisch
- Priorität: Sofort beheben

2. Cross-Site-Scripting (XSS) über die Suchfunktion

- | | | |
|--------------|------------|--|
| - | Betroffene | URL: |
| | | `http://juice-shop:3000/#/search?q=%3Ciframe%20src%3D%22javascript:alert(%60xss%60)%22%3E` |
| - Risiko: | Hoch | |
| - Priorität: | Sehr hoch | |

3. Fehlender Brute-Force-Schutz auf der Login-Seite

- Betroffene URL: `http://juice-shop:3000/#/login`
- Risiko: Mittel bis hoch
- Priorität: Hoch

4. Detailanalyse pro Schwachstelle

Schwachstelle 1: SQL-Injection mit Login-Bypass

- Betroffene Seite / URL:

`http://juice-shop:3000/#/login`

In den Rohdaten ist die Schwachstelle zwar einmal mit URL `http://juice-shop:3000/#/search` verknüpft, sie steht aber fachlich klar im Kontext des Login-Formulars und der Credentials. Daher ist die Login-Seite als primär betroffen anzusehen.

- Typ der Schwachstelle:

SQL Injection

- Technische Beobachtung:

In den Vulnerability-Daten wurde folgender Payload dokumentiert:

- Eingabefeld: `email`
- Payload: `` OR 1=1 --`
- Beobachtung: `Login bypass via SQL injection`

Zusätzlich wurden Credentials gespeichert, die auf einen erfolgreichen Authentifizierungsvorgang

hindeuten:

- Benutzer: `admin@juice-sh.op`
- Token/JWT wurde ausgegeben
- Rolle im Token: `admin`

Diese Kombination ist ein starker Hinweis darauf, dass die SQL-Injection nicht nur theoretisch vorhanden ist, sondern praktisch zu einem Admin-Login ohne legitime Zugangsdaten geführt hat.

- Mögliche Auswirkungen:
- Umgehung der Authentifizierung
- Unbefugter Zugriff auf Benutzerkonten
- Vollständige administrative Kompromittierung
- Zugriff auf sensible Daten
- Manipulation oder Löschung von Daten
- Möglicherweise weitergehende Datenbankkompromittierung

- Geschätzte CVSS 4.0-Bewertung:

9.3-9.8 (Kritisch), konservativ geschätzt: 9.4 / Critical

Begründung:

- Angreifbar über das Netzwerk
- Geringe Angriffskomplexität
- Keine reguläre Authentifizierung nötig
- Direkter Einfluss auf Vertraulichkeit, Integrität und potenziell Verfügbarkeit
- Nachweis eines erfolgreichen Admin-Zugriffs

Einschränkung:

Die exakte CVSS-4.0-Vektorermittlung ist aus den Rohdaten nicht vollständig ableitbar, etwa hinsichtlich aller nachgelagerten Systemauswirkungen. Die Bewertung bleibt daher eine fundierte Schätzung.

- Priorität:

Sofort / Kritisch

- Empfohlene Gegenmaßnahmen:
- Alle Datenbankzugriffe konsequent auf parametrisierte Queries / Prepared Statements umstellen
- Niemals Benutzereingaben direkt in SQL-Abfragen einfügen
- Serverseitige Eingabevalidierung ergänzen, aber nicht als alleinigen Schutz betrachten
- Authentifizierungslogik und ORM-/Datenbankabfragen überprüfen
- Sicherheitsrelevante Logs auswerten, um Missbrauch festzustellen

- Alle administrativen Tokens/Sessions invalidieren
- Betroffene Zugangsdaten und Geheimnisse rotieren
- Regressionstests für SQL-Injection in Login- und API-Endpunkten einführen

- Leicht verständliche Erklärung:

Hier kann sich ein Angreifer offenbar einloggen, ohne das richtige Passwort zu kennen. Durch eine manipulierte Eingabe wird die Datenbank so ausgetrickst, dass sie den Login trotzdem akzeptiert. Im schlimmsten Fall erhält der Angreifer sogar Administratorrechte.

Schwachstelle 2: Cross-Site-Scripting (XSS) über die Suchfunktion

- Betroffene Seite / URL:

`http://juice-shop:3000/#/search?q=%3Ciframe%20src%3D%22javascript:alert(%60xss%60)%22%3E`

- Typ der Schwachstelle:

Cross-Site Scripting (XSS), wahrscheinlich reflektiert oder DOM-basiert

- Technische Beobachtung:

In den Rohdaten steht:

- Typ: `XSS Injection`
- Parameter/Feld: `search-input`
- Payload: `
&amp;amp;lt;li&amp;amp;gt;- Beobachtung: `Alert message is triggered at input`&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;/ul&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="72 627 935 667" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;Das ist ein deutlicher Hinweis darauf, dass Eingaben aus der Suchfunktion im Browser unsicher verarbeitet und als aktiver Code interpretiert werden.&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="72 687 289 704" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;- Mögliche Auswirkungen:&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="72 711 557 846" data-label="List-Group"&amp;amp;gt;
&amp;amp;lt;ul&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;- Ausführung von JavaScript im Browser des Benutzers&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;- Diebstahl von Sitzungsinformationen, falls zugänglich&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;- Manipulation von Seiteninhalten&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;- Phishing innerhalb der legitimen Website&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;- Ausführung von Aktionen im Kontext des Benutzerkontos&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;li&amp;amp;gt;- Reputationsschaden und Vertrauensverlust&amp;amp;lt;/li&amp;amp;gt;
&amp;amp;lt;/ul&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="72 865 367 882" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;- Geschätzte CVSS 4.0-Bewertung:&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="72 888 413 906" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;7.1-8.0, konservativ geschätzt: 7.5 / High&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="72 925 183 943" data-label="Text"&amp;amp;gt;
&amp;amp;lt;p&amp;amp;gt;Begründung:&amp;amp;lt;/p&amp;amp;gt;
&amp;amp;lt;/div&amp;amp;gt;
&amp;amp;lt;div data-bbox="471 961 524 976" data-label="Page-Footer"&amp;amp;gt;Seite 5&amp;amp;lt;/div&amp;amp;gt;

- Ausnutzbar über das Netzwerk
- Geringe technische Hürde
- Typischerweise Benutzerinteraktion nötig (Aufruf des manipulierten Links)
- Hohe Auswirkungen auf Vertraulichkeit und Integrität im Browser-Kontext möglich

Einschränkung:

Aus den Rohdaten geht nicht eindeutig hervor, ob die Schwachstelle reflektiert, gespeichert oder rein DOM-basiert ist und ob zusätzliche Browser-Schutzmechanismen wirken. Daher vorsichtige Bewertung.

- Priorität:

Sehr hoch

- Empfohlene Gegenmaßnahmen:

- Suchparameter niemals ungefiltert in HTML oder DOM ausgeben
- Kontextgerechtes Output-Encoding einsetzen
- Gefährliche HTML-/JavaScript-Konstrukte server- und clientseitig unterbinden
- Verwendung sicherer Framework-Mechanismen zur DOM-Manipulation
- Content Security Policy (CSP) einführen oder verschärfen
- Sicherheitstests speziell für Such-, Kommentar- und Chat-Eingaben ergänzen

- Leicht verständliche Erklärung:

Ein Angreifer kann offenbar einen manipulierten Suchbegriff einbauen, der im Browser nicht als Text, sondern als ausführbarer Code behandelt wird. Dadurch könnte die Website für Besucher unbemerkt schädliche Aktionen ausführen.

Schwachstelle 3: Fehlender Schutz gegen Brute-Force-Angriffe

- Betroffene Seite / URL:

`http://juice-shop:3000/#/login`

- Typ der Schwachstelle:

Fehlender oder unzureichender Brute-Force-Schutz / Login Rate Limiting

- Technische Beobachtung:

In den Rohdaten wurde vermerkt:

- `missing\_bruteforce\_protection`
- Payload/Test: `40 failed login attempts`

- Beobachtung:

`Keine Änderung von Statuscode, Response-Body oder Retry-After nach 40 fehlgeschlagenen Login-Versuchen`

Das deutet darauf hin, dass die Anwendung keine wirksamen Gegenmaßnahmen wie Sperren, Verzögerungen, Captcha-Nachschaltung oder Rate Limiting implementiert hat.

- Mögliche Auswirkungen:

- Automatisiertes Durchprobieren von Passwörtern
- Übernahme schwacher Benutzerkonten
- Erhöhtes Risiko bei Passwort-Wiederverwendung
- Unterstützung weiterer Angriffe auf privilegierte Konten

- Geschätzte CVSS 4.0-Bewertung:

6.0-6.8, konservativ geschätzt: 6.4 / Medium

Begründung:

- Über das Netzwerk ausnutzbar
- Keine oder geringe Angriffskomplexität
- Kein direkter Sofortschaden ohne passende Zugangsdaten
- Risiko hängt stark von Passwortqualität, MFA und Monitoring ab

Da aus den Rohdaten keine MFA oder Lockout-Mechanismen ersichtlich sind, ist die Schwachstelle relevant, aber weniger gravierend als der bestätigte SQL-Bypass.

- Priorität:

Hoch

- Empfohlene Gegenmaßnahmen:

- Rate Limiting pro IP, Konto und Geräteprofil
- Temporäre Sperren oder progressive Verzögerungen nach Fehlversuchen
- Erkennung verteilter Login-Angriffe
- MFA für privilegierte und möglichst für alle Benutzerkonten
- Monitoring und Alarmierung bei ungewöhnlichen Login-Mustern
- Einheitliche Fehlermeldungen ohne Informationsleck

- Leicht verständliche Erklärung:

Die Login-Seite bremst wiederholte falsche Anmeldeversuche offenbar nicht ausreichend aus. Dadurch können Angreifer automatisiert viele Passwortkombinationen ausprobieren, bis sie Erfolg haben.

5. Konkrete Handlungsempfehlungen

Sofortmaßnahmen

1. SQL-Injection umgehend beheben

- Authentifizierungslogik prüfen
- Prepared Statements erzwingen
- Alle Sessions und Tokens zurücksetzen
- Admin-Konten und sensible Konten auf Missbrauch prüfen

2. XSS in der Suche beseitigen

- Unsichere Ausgabe des Suchparameters unterbinden
- Encoding/Sanitizing korrekt nach Ausgabekontext umsetzen
- CSP ergänzen

3. Login-Schutz erhöhen

- Rate Limiting
- Lockout/Backoff-Strategien
- MFA insbesondere für Admin-Zugänge

Kurzfristige Maßnahmen

4. Sicherheitsreviews für alle Eingabewege durchführen

- Suche
- Login
- Kontaktformular
- Beschwerdeformular
- Chatbot
- Registrierungs- und Passwort-Reset-Funktionen
- Upload-Funktionen auf `photo-wall` und `complain`

5. Automatisierte Sicherheitstests erweitern

- Regressionstests für SQLi und XSS
- Negative Tests für Authentifizierung und Session-Handling
- Abuse-Case-Tests für Brute-Force und Account-Takeover

6. Monitoring und Incident Response prüfen

- Fehlgeschlagene Logins zentral protokollieren
- Auffällige Authentifizierungsereignisse alarmieren
- Prüfung, ob der dokumentierte Admin-Login missbräuchlich nachvollzogen werden kann

Mittelfristige Maßnahmen

7. Secure Development Practices stärken

- Security Code Review
- Bedrohungsmodellierung für Authentifizierung und Eingabeverarbeitung
- Sichere Standardbibliotheken für DB-Zugriffe und Output-Encoding

8. Upload- und Formularbereiche gezielt nachtesten

- In den Rohdaten sind Dateiuploads und mehrere dynamische Formulare sichtbar
- Dafür liegt zwar kein bestätigter Befund vor, aber das Angriffspotenzial ist grundsätzlich erhöht

6. Gesamtfazit

Die analysierte Anwendung weist mindestens eine bestätigte kritische Authentifizierungsschwachstelle und eine bestätigte clientseitige Code-Injection auf. Besonders schwerwiegend ist, dass die SQL-Injection offenbar bereits einen Admin-Login ohne reguläre Berechtigung ermöglicht hat. Dadurch ist das Risiko einer vollständigen Kompromittierung als sehr hoch einzustufen.

Die zusätzlich fehlende Brute-Force-Abwehr verschärft die Lage, da auch klassische Passwortangriffe erleichtert werden.

Andere untersuchte Seiten enthalten laut den vorliegenden Rohdaten zwar Formulare und potenziell interessante Angriffsflächen, aber keine bestätigten zusätzlichen Schwachstellen. Diese Bereiche sollten dennoch gezielt manuell oder vertieft automatisiert nachgeprüft werden.

Gesamtbewertung:

Die Sicherheitslage ist aktuell kritisch, insbesondere wegen der bestätigten SQL-Injection mit Login-Bypass. Die Behebung sollte unverzüglich priorisiert werden.